

Pourquoi un array Numpy est plus rapide qu'une liste Python ?

Septembre 2026

Aubin GEOFFRE

SISDev, Centre des Matériaux, MINES Paris

- ▶ On calcule un tableau formé des carrés des 10 000 premiers entiers
- ▶ On compare Numpy et **Python natif**
- ▶ En moyenne, Python natif est ~ **68 fois plus lent**

```
1 %%timeit
2 L = []
3 for i in range(10000):
4     L.append(i*i)
```

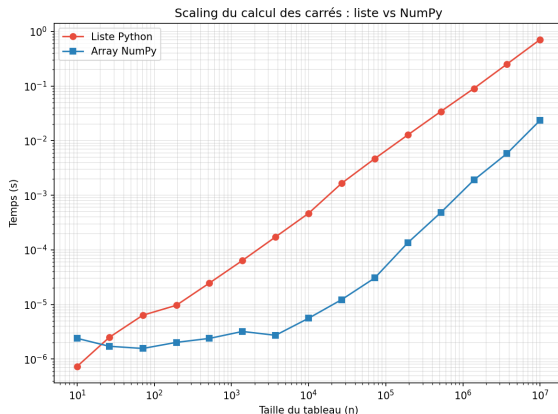
946 μ s $\pm$ 267 μ s per loop (mean $\pm$ std. dev. of 7 runs, 1000 loops each)

```
1 %%timeit
2 import numpy as np
3 arr = np.square(np.arange(10000))
```

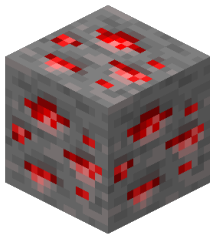
16.4 μ s $\pm$ 3.83 μ s per loop (mean $\pm$ std. dev. of 7 runs, 100000 loops each)

NUMPY VS PYTHON

- ▶ Ce constat se **généralise**
- ▶ Plus le tableau est **grand**, plus l'écart **se creuse**
- ▶ On atteint des temps de calcul rapidement **problématiques** lorsqu'on reste en Python natif
- ▶ **Pourquoi ?**



Une histoire de 0 et 1



ÉCRITURE BINAIRE : GÉNÉRALITÉS

- ▶ L'informatique repose sur un **système binaire** : 1 (présence) ou 0 (absence)
- ▶ **1 bit** : 0 ou 1
- ▶ On peut décomposer tout nombre entier n de façon unique :

$$n = \sum_{i=0}^{N-1} b_i 2^i = b_0 \cdot 1 + b_1 \cdot 2 + b_2 \cdot 4 + \cdots + b_{N-1} \cdot 2^{N-1}$$

avec $b_i \in \{0, 1\}$ et N le nombre de bits.

- ▶ Ex :

$$5 = 1 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 = 1 + 0 + 4$$

- ▶ Le tuple (101) est équivalent à 5
- ▶ La connaissance du tuple $(b_{N-1} \cdots b_0)$ donne une **représentation binaire** équivalente

EXERCICE : BINAIRE

- ▶ Calculer la représentation binaire de $n = 3030$
- ▶ Vérifier avec `bin()`
- ▶ La retransformer en décimal

EXERCICE : BINAIRE

- ▶ Calculer la représentation binaire de $n = 3030$
- ▶ Vérifier avec `bin()`
- ▶ La retransformer en décimal

```
1 n = 3030
2 print(bin(n))
3
4 binary = ""
5 while (n != 0):
6     reste = n % 2
7     binary = str(reste) + binary
8     n = n // 2
9
10 res = 0
11 for (i, bit) in enumerate(binary) :
12     N = len(binary)
13     res += int(bit) * 2**(N - i - 1)
14 print(res)
```

```
0b101111010110
3030
```

ÉCRITURE BINAIRE : PROFONDEUR DE REPRÉSENTATION

- ▶ Le nombre de chiffres représentables croît **exponentiellement** avec le nombre de bits
 - 1 bit $\rightsquigarrow$ 0, 1 $\rightsquigarrow$ 2 valeurs
 - 2 bits $\rightsquigarrow$ 0, 1, 2, 3 $\rightsquigarrow$ 4 valeurs
 - 3 bits $\rightsquigarrow$ 0, 1, $\dots$, 7 $\rightsquigarrow$ 8 valeurs
 - N bits $\rightsquigarrow$ 2^N valeurs
 - ▶ Il existe un choix de représentation selon le **signe**
 - $0, \dots, 2^N - 1$ (**unsigned**)
 - $-2^{N-1}, \dots, 2^{N-1} - 1$ (**signed**)
 - ▶ Exemple sur **8 bits**
 - Unsigned : $0, \dots, 255$ (`np.uint8`)
 - Signed : $-128, \dots, 127$ (`np.int8`)
 - ▶ Impossible de travailler avec un entier supérieur à 255 avec 8 bits : au delà on a un **overflow**
- $\rightsquigarrow$ Le nombre de bits conditionne la **profondeur de représentation**

```
1 x = np.uint8(255)
2 print(x)
3 print(x + 1)
```

```
255
0
/tmp/ipykernel_740/2462590373.py:3: RuntimeWarning: overflow encountered in scalar add
  print(x + 1)
```


EXERCICE : PROFONDEUR

- ▶ Combien de valeurs peut encoder un `np.int16` ? Lesquelles ? Quelle taille en octet ? Et pour un `np.uint32` ?
- ▶ Vérifier cela en provoquant un overflow Numpy

EXERCICE : PROFONDEUR

- ▶ Combien de valeurs peut encoder un `np.int16` ? Lesquelles ? Quelle taille en octet ? Et pour un `np.uint32` ?
- ▶ Vérifier cela en provoquant un overflow Numpy
- ▶ `np.int16` : 16 bits, 2 octets, de $-2^{16-1} = -32768$ à $2^{16-1} - 1 = 32767$
- ▶ `np.uint32` : 32 bits, 4 octets, de 0 à $2^{32} - 1 = 4294967295$

- ▶ **1 octet = 1 Byte = 8 bits**
- ▶ On ne peut pas adresser moins de 1 octet
- ▶ Historiquement tout le monde s'est aligné sur cette convention
- ▶ **Pratique** d'un point de vue arithmétique binaire (c'est une puissance de 2 !)
- ▶ Le passage à 64 bits a été révolutionnaire : l'adressage mémoire explose de 2^{32} à 2^{64} possibilités

Octets	Bits	Valeurs possibles (2^N)	Type Numpy (non signé)
1	8	256	<code>np.uint8</code>
2	16	65 536	<code>np.uint16</code>
4	32	$\sim 4,3 \times 10^9$	<code>np.uint32</code>
8	64	$\sim 1,8 \times 10^{19}$	<code>np.uint64</code>

NOMBRES FLOTTANTS

- ▶ Comment représenter un réel avec des nombres entiers ?
- ▶ Avec l'**écriture scientifique** !
- ▶ On peut approximer un nombre réel x par

$$x \approx (-1)^s \cdot m \cdot 10^E$$

$$\pi \approx (-1)^0 \cdot 31415 \cdot 10^{-4}$$

avec s le bit de signe (1 bit), m la mantisse et E l'exposant¹

- ▶ Le tuple (s, m, E) composé de trois **entiers** suffit à décrire le réel
- ▶ Il s'agit de la **représentation flottante** des réels
- ▶ La norme **IEEE 754** décrit la représentation standard
 - **Simple précision** (32 bits) : 8 bits d'exposant, 23 bits de mantisse
 - **Double précision** (64 bits) : 11 bits d'exposant, 52 bits de mantisse
- ▶ Un double est plus précis, plus profond mais plus lourd

1. En pratique, on exprime le résultat en base 2 : $x \approx (-1)^s \cdot m \cdot 2^E$, ce qui ne change pas grand chose d'un point de vue philosophique

EXERCICE : APPROXIMATION

- 1 Faire $0.1 + 0.2 == 0.3$: que se passe-t-il ?
- 2 Faire $1e16 + 1 - 1e16$: que se passe-t-il ? Et $1e16 - 1e16 + 1$?

EXERCICE : APPROXIMATION

- 1 Faire $0.1 + 0.2 == 0.3$: que se passe-t-il ?
- 2 Faire $1e16 + 1 - 1e16$: que se passe-t-il ? Et $1e16 - 1e16 + 1$?
- 1 0.1 s'écrit en binaire $0.00011001100110011\dots$: la représentation flottante effectue une troncature donc une légère approximation. Cette approximation se répercute sur le résultat final :
 0.30000000000000004
- 2 Compte tenu de l'approximation flottante, la précision n'est pas suffisante pour capter le 1 de $1e16 + 1 = 10\dots01$

EXEMPLES DE TYPES PYTHON ET NUMPY

► En Python natif

- `float` : double précision (64 bits)
- `int` : ça dépend !
- Les types Python natifs sont plus lourds que prévu : ce sont des **objets complets** (`PyObject`), pas juste la donnée brute

► En Numpy

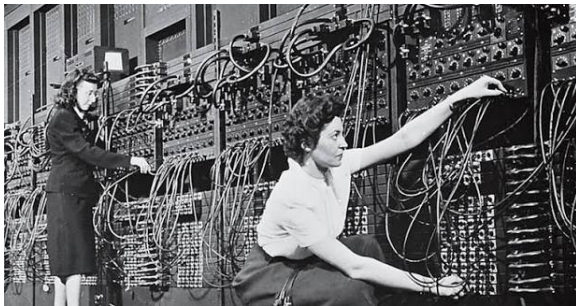
- `np.uint8` : entier sur 8 bits non-signé
- `np.int16` : entier sur 16 bits signé
- `np.float32` : simple précision
- `np.float64` : double précision

► Les caractères passent par une correspondance entre un entier et un symbole (**ASCII**, Unicode)

```
1 print("Size np.uint8 : ", np.dtype(np.uint8).itemsize)
2 print("Size np.int16 : ", np.dtype(np.int16).itemsize)
3 print("Size np.float64 : ", np.dtype(np.float64).itemsize)
4 print("Size int 0 : ", sys.getsizeof(0))
5 print("Size float 3.14 : ", sys.getsizeof(3.14))
```

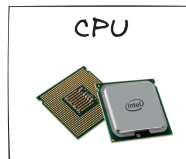
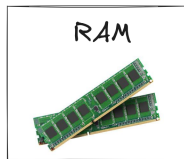
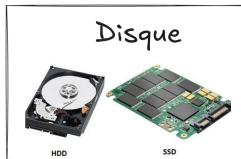
```
Size np.uint8 : 1
Size np.int16 : 2
Size np.float64 : 8
Size int 0 : 28
Size float 3.14 : 24
```

Mais au fait c'est quoi un ordinateur ?



UN ORDI EN BREF

- ▶ Pour le moment, nous n'avons parlé seulement maths
- ▶ En gros, votre ordinateur (smartphone, console, portable,...) est composé de :
 - Central Process Unit (**CPU**)
 - Mémoire vive / Random Access Memory (**RAM**)
 - Stockage lent (**disque**)
- ▶ D'autres éléments sont importants mais pas abordés ici
 - Carte graphique (**GPU**) (~ calculateur spécialisé dans un certain type d'opération)
 - Système d'exploitation (**OS**) : Windows, macOS, Linux²
 - Périphériques (clavier, souris, etc)



2. Pour en finir avec une idée reçue, l'OS le plus utilisé au monde n'est ni Windows ni macOS, mais **Android**, qui repose sur un **noyau Linux** !

CPU : GÉNÉRALITÉS

- ▶ C'est le **calculateur** de votre ordinateur
- ▶ Extrêmement **rapide** mais extrêmement **peu** de **mémoire**
- ▶ Sa **cadence** est exprimée en GHz *i.e.* nombre d'opération à la seconde
- ▶ 3 GHz $\rightsquigarrow$ 3 milliards d'opérations par seconde / **1 opération en 3 ns !!!**
- ▶ En fait, il passe beaucoup de son temps à **attendre** la donnée
- ▶ Une opération CPU est représentée par une ligne d'**assembleur**³
- ▶ Fabricants : Intel, AMD, etc



3. Le langage assembleur est le dernier langage lisible par l'humain avant le binaire pur : il décrit les opérations élémentaires exécutées à l'échelle du CPU

CPU : MÉMOIRE ET HIÉRARCHIE

- ▶ Un CPU possède **2 mémoires différentes**

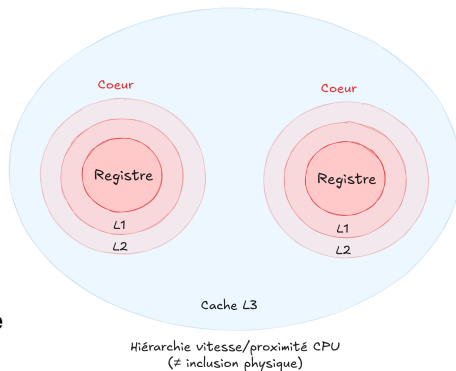
1 Registres

- Mémoire : ~ 1 ko, rapidité : < 1 ns
- Directement dans le CPU
- Les valeurs sont manipulées ici
- Doit être alimenté en mémoire de l'extérieur

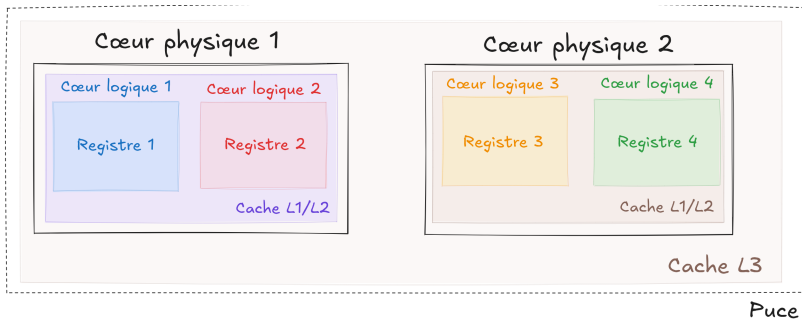
2 Caches

- L1 : ~ 50 ko, par cœur, ~ 1 ns
- L2 : ~ 500 ko, par cœur, ~ 5 ns
- L3 : quelques Mo, **partagé entre cœurs**, ~ 15 ns

- ▶ Plus la mémoire est **rapide**, plus elle est **chère et petite**

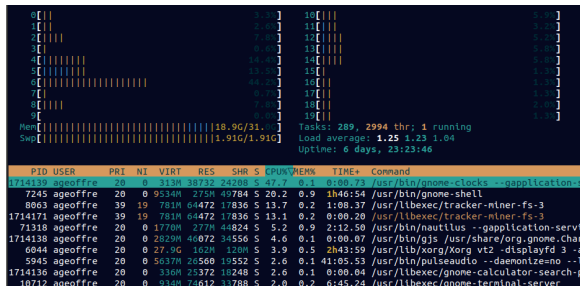


- ▶ Un cœur = 1 **calculateur indépendant** avec ses propres registres et caches L1/L2
- ▶ Nativement Python utilise un unique cœur
- ▶ Numpy sait utiliser **plusieurs cœurs**
- ▶ Deux types de cœurs
 - Cœurs **physiques** : calculateur réel
 - Cœurs **logiques (thread matériel)** : chaque cœur physique peut gérer plusieurs flux d'instruction (généralement 2), en partageant ses ressources de calcul (**hyper-threading**)



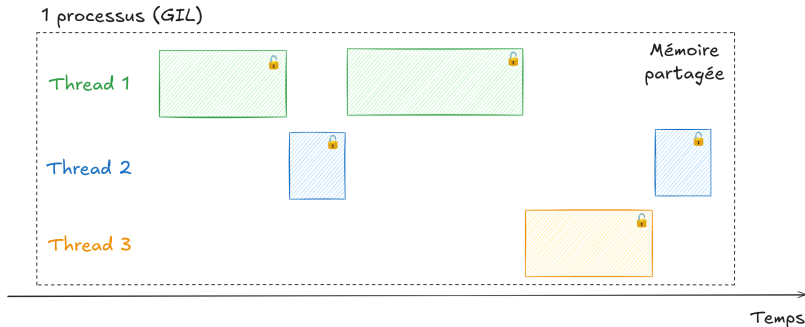
PROCESSUS ET THREADS

- ▶ 1 exécution de programme (e.g. `python fichier.py`) = 1 **processus** avec une mémoire isolée
- ▶ Au sein d'un process, plusieurs **threads logiciels** peuvent travailler en partageant la mémoire
 - Communication entre processus : **cher** (mémoire séparée)
 - Communication entre threads : **rapide** (mémoire commune) mais **risque** de modifier la même mémoire !
- ▶ L'OS **répartit** les processus / threads sur les cœurs logiques à disposition (**scheduling**)
- ▶ Sur N cœurs logiques, on peut répartir N threads
- ▶ Au-delà, les threads se relaient rapidement sur les cœurs disponibles

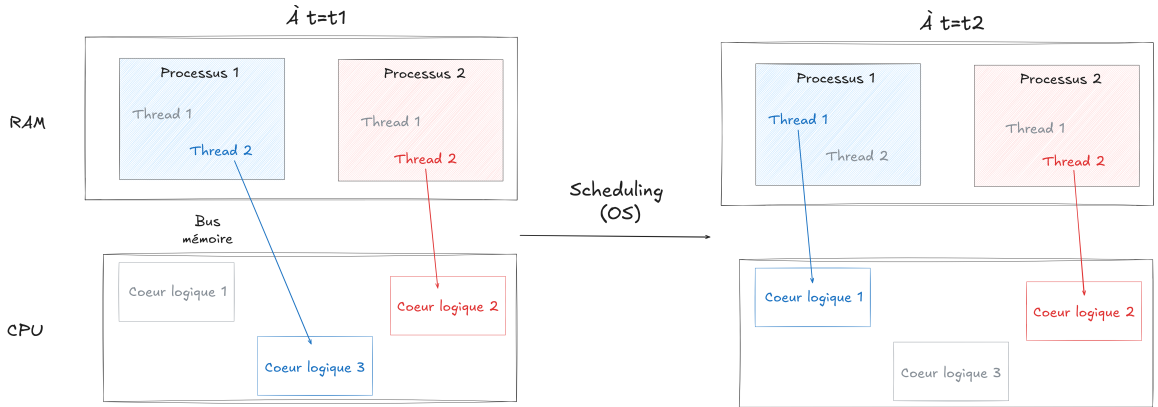


THREAD-SAFETY ET GIL

- ▶ Les threads partagent la mémoire : ils peuvent modifier la même donnée au même moment
- ▶ Opération **thread-safe** : correcte quand exécutée sur plusieurs threads
- ▶ Si elle ne l'est pas, le résultat dépend de l'ordre d'exécution (**race condition**)
- ▶ En Python, par défaut, 1 processus = 1 **GIL** (Global Interpreter Lock)
- ▶ Un GIL force qu'un unique thread travaille à l'instant t , quand bien même plusieurs peuvent être créés
- ▶ Il est possible de faire du parallélisme en Python : `multiprocessing` (plusieurs processus, chacun son GIL), déléguer à du **code C**,... (comme Numpy)

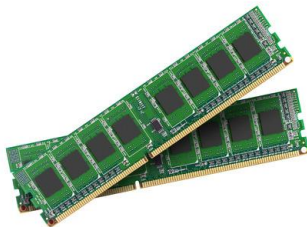


INTERACTION CPU-RAM DANS UN CONTEXTE GIL



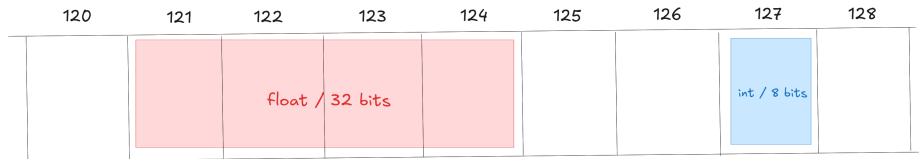
RAM : GÉNÉRALITÉS

- ▶ C'est la **mémoire de travail** de l'ordinateur
- ▶ Plus **grande** ($\sim 10\text{-}100$ Go) mais aussi beaucoup plus **lente** (~ 100 ns)
- ▶ Il y a un facteur de rapidité de **10 à 100**
- ▶ C'est là où vivent les **variables** pendant l'exécution d'un programme
- ▶ La RAM alimente les caches, qui alimentent le calculateur du CPU



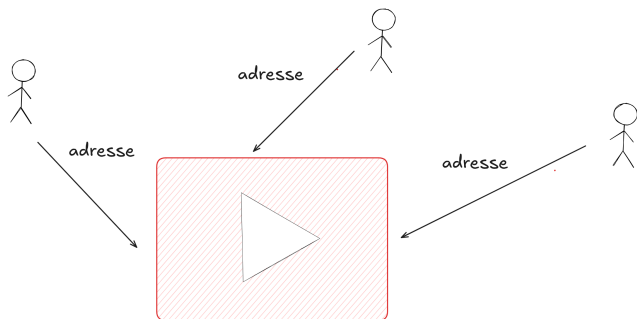
RAM : ORGANISATION DE LA MÉMOIRE

- ▶ On peut visualiser la RAM comme un **ruban** divisé en **cases** régulières
- ▶ 1 case = 1 octet
 - Un entier sur 8 bits $\rightsquigarrow$ 1 case
 - Un float sur 64 bits $\rightsquigarrow$ 8 cases
- ▶ Le numéro de la case est une **adresse mémoire**
- ▶ La donnée est stockée à cette adresse
- ▶ **Contiguïté mémoire** : les données de l'objet se suivent en mémoire (*i.e.* cases adjacentes)
- ▶ Si les données sont contiguës, leur accès est **très rapide** car elles sont regroupées au même endroit



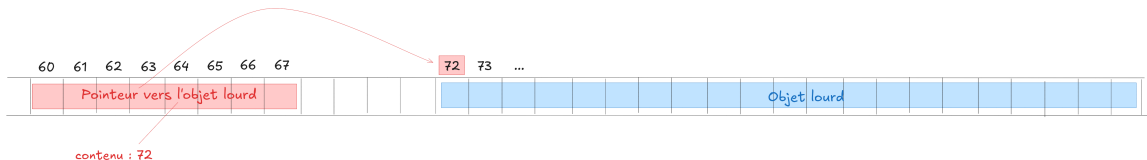
POURQUOI LES POINTEURS ?

- ▶ Imaginons que vous ayez vu une super vidéo d'1h que vous souhaitez envoyer à un ami
- ▶ Vous n'allez pas lui envoyer la vidéo en entier mais un **lien** vers cette vidéo
- ▶ Personne n'a envie de manipuler un objet aussi lourd
- ▶ En allant à la bonne **adresse**, il aura accès au contenu
- ▶ Jamais la vidéo n'aura été déplacée
- ▶ Pour le même résultat, vous aurez déplacé **30 octets au lieu d'1 Go**
- ▶ C'est le principe d'un **pointeur**



POINTEUR

- ▶ Un **pointeur** stocke l'adresse mémoire d'une autre variable
- ▶ C'est la base de certains langages (C, C++)
- ▶ Python masque les pointeurs à l'utilisateur : **ils sont pourtant bien présents !**
- ▶ Un pointeur traduit un entier : c'est un objet **ultra léger**
- ▶ Peu importe le type pointé, son poids est identique, généralement **8 octets** (64 bits)
- ▶ Un objet léger peut être moins lourd que son pointeur !



NOTATION DES ADRESSES

- ▶ Par commodité, les adresses sont représentées en base $16 = 2^4$ (**hexadécimal**)

$$n = \sum_i a_i 16^i$$

avec $a_i \in [[0, \dots, 15]]$

- ▶ Par convention, les nombres entre **10 et 15** sont notés par des **lettres**

$$a \leftrightarrow 10$$

$$b \leftrightarrow 11$$

et ainsi de suite !

- ▶ Pour indiquer que la notation est hexadécimale, l'adresse commence par **0x**
- ▶ Une adresse peut être `0x2cb7d3d0`
- ▶ Cela traduit bien un entier

<code>0x303d</code>	<code>0x303e</code>	<code>0x303f</code>	<code>0x3040</code>	<code>0x3041</code>

ADRESSAGE EN PYTHON

- ▶ L'adresse d'une variable est accessible avec `id()`
- ▶ Elle peut être convertie en hexadécimal avec `hex()`
- ▶ Dans l'exemple `a` et `b` partagent la même adresse $\rightsquigarrow$ c'est un nom différent pour la même chose (**alias**)
- ▶ On dit que `b` est une **référence** à `a`
- ▶ L'égalité des pointeurs peut être testée avec `is` alors que `==` teste l'égalité des valeurs
- ▶ Dans 99% des cas, Python ne copie pas les **données brutes**, mais copie leur **adresse** (**passage par référence**)

```
1 a = 200.  
2 b = a  
3 print(hex(id(a)))  
4 print(hex(id(b)))  
5 print(a == b)  
6 print(a is b)
```

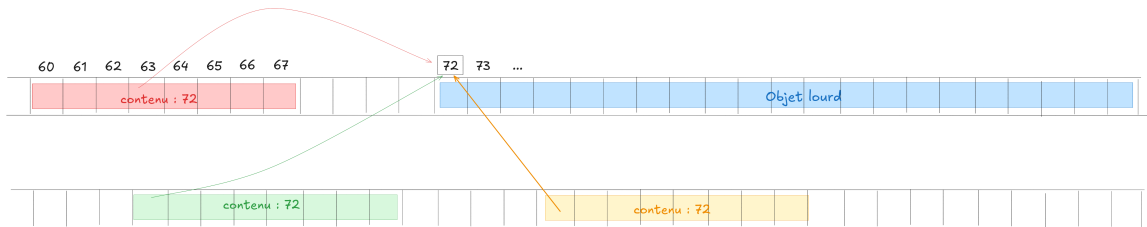
```
0x7821b1950550  
0x7821b1950550  
True  
True
```

INTÉRÊT DES POINTEURS

► Les pointeurs ont de **nombreuses utilités**

- Ne pas manipuler les données (potentiellement volumineuses) mais un objet léger équivalent : les données précieuses ne sont pas copiées
- Avoir des objets pour lesquels on ne connaît pas la taille à l'avance (**allocation dynamique**)
- Plusieurs variables peuvent partager le même objet sans duplication des données

► C'est sur ces principes que repose la **liste Python** !



MÉMOIRE DISQUE

- ▶ Les mémoires décrites précédemment sont dites **volatiles**
- ▶ Elles ne survivent pas au fait d'éteindre la machine⁴
- ▶ On a besoin d'une **mémoire persistante** : mémoire **disque**
- ▶ Différentes technologies (HDD, SSD)
- ▶ **Très lente** ($\sim 0.1-10$ ms) mais **abondante** (> 100 Go)

SSD



Solid State Drive

HDD

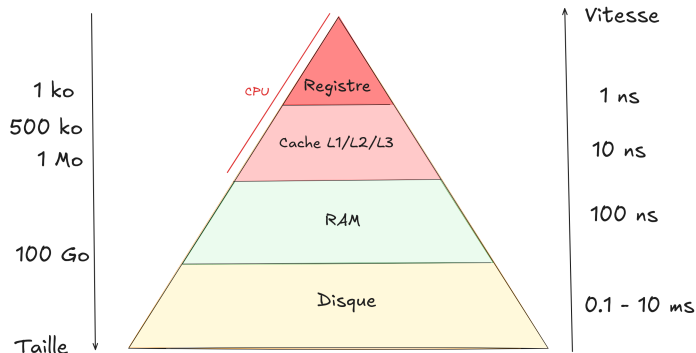


Hard Disk Drive

4. Oui, redémarrer une machine permet de vider la RAM !

BILAN SUR LA MÉMOIRE

- ▶ **Plus la mémoire est rapide, plus elle est chère et petite**
- ▶ **Lorsque les données sont regroupées en mémoire, leur accès est plus rapide**
- ▶ On représente généralement la mémoire avec une pyramide
- ▶ Ceci explique la rapidité de Numpy en comparaison à Python natif



CACHE HIT / CACHE MISS

- ▶ Le CPU a besoin de la donnée pour calculer
- ▶ Deux possibilités
 - La donnée demandée est présente dans le cache (**cache hit**) $\leadsto$ accès rapide
 - La donnée demandée est absente du cache (**cache miss**) $\leadsto$ on va la chercher dans la RAM puis la copier dans le cache
- ▶ Un miss coûte ~ 10 à 100 fois plus cher qu'un hit
- ▶ Le cache fonctionne par **blocs** : chaque chargement ramène ~ 64 octets d'un coup
- ▶ Si les données sont **contiguës**, un chargement peut couvrir plusieurs éléments utiles (*i.e.* moins de misses)
- ▶ Numpy repose sur cette idée pour accélérer ses calculs

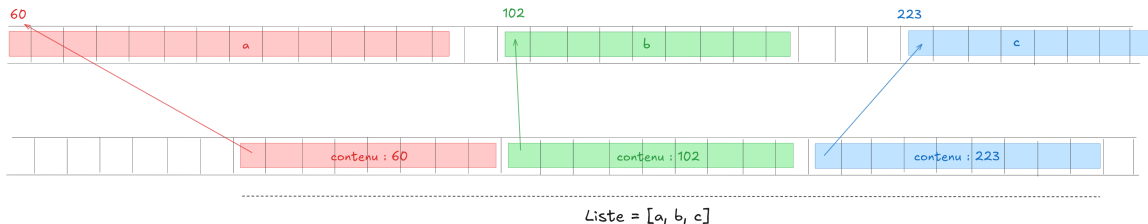


Liste vs. Numpy array



C'EST QUOI UNE LISTE PYTHON ?

- ▶ Une **liste Python** est un conteneur dont la taille est **dynamique** (on peut ajouter/enlever des éléments) pouvant accueillir des **types hétérogènes**
- ▶ Derrière, une liste stocke les **adresses mémoire** des objets qu'elle contient⁵
- ▶ Les objets contenus sont **dispersés** en mémoire
- ▶ Cela explique sa lenteur !



5. ... mais les pointeurs sont contigus !

NUMPY SOUS LE CAPOT

- ▶ Numpy travaille avec des tableaux de **taille fixe** et de **type homogène** (`ndarray`)
- ▶ La taille est fixée à l'**initialisation** du tableau et ne changera plus
- ▶ Pour un tableau de 10 flottants à double précision, la taille sera $10 \times 8 = 80$ octets
- ▶ Les données sont **contiguës** en mémoire
- ▶ Attention, la taille mémoire (`.nbytes`) n'est pas la forme du tableau (`.shape`)

```
1 arr = np.arange(10, dtype=np.float64)
2 arr.nbytes, arr.shape
```

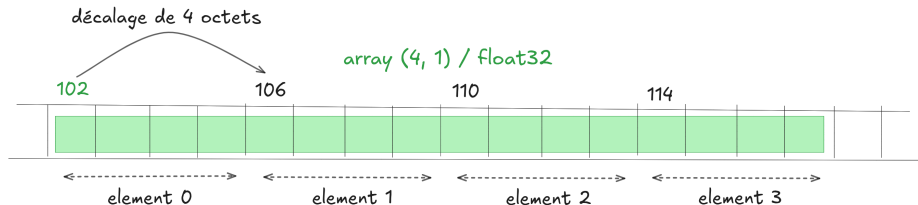
```
(80, (10,))
```

MANIPULATION DE TABLEAU NUMPY

- ▶ Les tableaux Numpy sont **contigus en mémoire (buffer)**
- ▶ Un **buffer** : zone mémoire contiguë contenant des données brutes
- ▶ Si on connaît l'adresse du premier élément et la taille d'un élément, on peut accéder à l'élément n

$$\text{Adresse}(n) = \text{Adresse}(0) + n \times \text{taille d'un élément}$$

- ▶ On opère simplement un **décalage** de n
- ▶ `arr[0]` est un décalage de 0 : il s'agit du **premier élément**
- ▶ Il s'agit d'une opération extrêmement **rapide** en mémoire



STRIDE

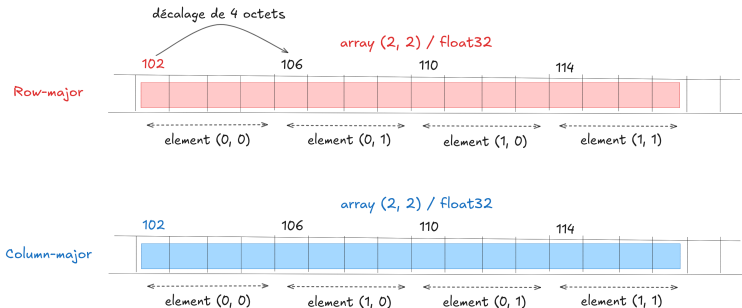
- Pour un **tableau 2D**, l'accès est un peu différent
- Deux choix : **column-major** (Fortran, colonnes contiguës) ou **row-major** (C, lignes contiguës)
- Pour récupérer (i,j) :

$$\text{Adresse}(i,j) = \text{Adresse}(0,0) + i \times \text{stride}_0 + j \times \text{stride}_1$$

- Row-major : $\text{stride}_0 = n_{\text{col}} \cdot S$, $\text{stride}_1 = S$
- Column-major : $\text{stride}_0 = S$, $\text{stride}_1 = n_{\text{row}} \cdot S$

avec S la taille d'un élément

- Le **stride** = décalage mémoire pour accéder à un élément



EXERCICE : STRIDES

- On prend un tableau 2×3 , chaque élément ayant une taille de 8 octets

$$\begin{pmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \end{pmatrix}$$

- **Row-major (par défaut sur Numpy)**

$$(a_{00} \quad a_{01} \quad a_{02} \quad a_{10} \quad a_{11} \quad a_{12})$$

$$\text{Adresse}(a_{11}) = \text{Adresse}(a_{00}) + 1 \times 3 \times 8 + 1 \times 8 = \text{Adresse}(a_{00}) + 32$$

Le stride est (24, 8).

- **Column-major**

$$(a_{00} \quad a_{10} \quad a_{01} \quad a_{11} \quad a_{02} \quad a_{12})$$

$$\text{Adresse}(a_{11}) = \text{Adresse}(a_{00}) + 1 \times 8 + 1 \times 2 \times 8 = \text{Adresse}(a_{00}) + 24$$

Le stride est (8, 16).

```
1 L = [[1, 2, 3], [4, 5, 6]]
2 arr_C = np.array(L, dtype=np.float64, order="C")
3 arr_F = np.array(L, dtype=np.float64, order="F")
4
5 arr_C.strides, arr_F.strides,
```

((24, 8), (8, 16))

Liste vs. Numpy array

Centre des Matériaux – A. GEOFFRE Septembre 2026

GÉNÉRALISATION

- ▶ Numpy **généralise** le principe précédent à des tableaux à n dimensions (`ndarray`)
- ▶ Un `ndarray` peut avoir autant d'indices qu'on veut
- ▶ Le principe est le même : les données sont contiguës en mémoire, seul le stride (et la forme) change
- ▶ Exemple pour une image RGB de 50×100 pixels
 - Une intensité de couleur prend **256 valeurs** $\rightsquigarrow$ `np.uint8`
 - On a un tableau ($50 \times 100 \times 3$)
 - Le stride devient un tuple (300, 3, 1)
 - La taille mémoire est $50 \times 100 \times 3 \times 1 = 15000$ octets

```
1 im = np.zeros((50,100,3), dtype=np.uint8)
2 print(im.shape)
3 print(im.strides)
4 print(im.nbytes)
```

```
(50, 100, 3)
(300, 3, 1)
15000
```


UNE FONCTION MAGIQUE

- ▶ L'injustement méconnue fonction `np.info` permet de lister toutes les informations que nous avons présentées
- ▶ Entre autres
 - La forme (`shape`)
 - Le stride (`strides`)
 - Le type contenu (`type`) et sa taille mémoire (`itemsize`)
 - La contiguïté (`contiguous`) et l'ordre de stockage (`fortran`)
 - L'adresse mémoire du premier élément du tableau (`data pointer`)

```
1 arr = np.zeros((2, 3), dtype=np.float32)
2 np.info(arr)
```

```
class: ndarray
shape: (2, 3)
strides: (12, 4)
itemsize: 4
aligned: True
contiguous: True
fortran: False
data pointer: 0x3c8c5b80
byteorder: little
byteswap: False
type: float32
```

INTÉRÊT DU STRIDE

- Suppose qu'on veuille transposer une matrice

$$\begin{pmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \end{pmatrix}$$

- On va pointer vers le tableau initial et **changer le stride**
- En row-major, le stride passe de (24, 8) à (8, 24)
- La forme passe de (2, 3) à (3, 2)
- Le coût de la transposition est quasiment **transparent**
- Le tableau et sa transposée partagent le même **emplacement mémoire**
- Donc modifier le tableau initial **modifie aussi** sa transposée

```
1 arr = np.arange(6, dtype=np.float64).reshape((2, 3))
2 arr_T = np.transpose(arr)
3 print(arr.strides, arr.ctypes.data)
4 print(arr_T.strides, arr_T.ctypes.data)
5 arr[0, 0] = 100
6 print(arr_T[0, 0])
```

```
(24, 8) 1031744848
(8, 24) 1031744848
100.0
```

- ▶ Numpy **déteste réallouer** de la mémoire inutilement
- ▶ Si la donnée utile existe quelque part, on ne va **pas** la recopier : ça ne sert à rien !
- ▶ À la place, Numpy joue sur les strides et les pointeurs
- ▶ À partir d'un tableau initial, on va faire des **vues** *i.e.* c'est le même tableau en mémoire mais **regardé différemment**
- ▶ C'est le cas du **slicing**⁶
- ▶ Exercice : prendre un array puis slicer du 2nd élément jusqu'à la fin. Comparer les adresses mémoire : qu'observe-t-on ?

6. ... sauf quand il n'y a pas de stride régulier possible : masque booléen, etc

- ▶ Numpy **déteste réallouer** de la mémoire inutilement
- ▶ Si la donnée utile existe quelque part, on ne va **pas** la recopier : ça ne sert à rien !
- ▶ À la place, Numpy joue sur les strides et les pointeurs
- ▶ À partir d'un tableau initial, on va faire des **vues** *i.e.* c'est le même tableau en mémoire mais **regardé différemment**
- ▶ C'est le cas du **slicing**⁶
- ▶ Exercice : prendre un array puis slicer du 2nd élément jusqu'à la fin. Comparer les adresses mémoire : qu'observe-t-on ?

```
1 arr = np.arange(10, dtype=np.uint8)
2 arr_sliced = arr[1:]
3 print(arr.ctypes.data)
4 print(arr_sliced.ctypes.data) # Adresse précédente + 1 octet
```

```
1038516512
1038516513
```

6. ... sauf quand il n'y a pas de stride régulier possible : masque booléen, etc

- ▶ **On ne fait jamais de boucle `for` sur un Numpy array !!!**
- ▶ Les fonctions Numpy **opèrent sur des tableaux** et non sur des éléments individuels
- ▶ Pourquoi ?
 - Avec un `for`, on ne tire **pas** parti de la contiguïté mémoire
 - Connaissant l'adresse de base et le stride, Numpy accède aux données **sans détour**
 - Les données étant contiguës, elles sont **préchargées** dans le cache du processeur
 - C'est le principe du **SIMD** (Single Instruction Multiple Data)
- ▶ Il existe une fonction Numpy **vectorisée** (`ufunc`) pour quasiment tout

```
1 arr = np.arange(10)
2 np.square(arr)
3 np.exp(arr)
4 print(np.exp)
```

<ufunc 'exp'>

EXERCICE : VECTORISATION

- Soient 2 vecteurs $\mathbf{a}$ et $\mathbf{b}$. Calculer la distance euclidienne :

$$\|\mathbf{a} - \mathbf{b}\| = \sqrt{\sum_i (a_i - b_i)^2}$$

- (1) en utilisant les `list` Python, (2) de façon vectorisée avec Numpy.
- Comparer les temps de calcul

EXERCICE : VECTORISATION

- Soient 2 vecteurs a et b . Calculer la distance euclidienne :

$$\|a - b\| = \sqrt{\sum_i (a_i - b_i)^2}$$

- (1) en utilisant les `list` Python, (2) de façon vectorisée avec Numpy.
- Comparer les temps de calcul

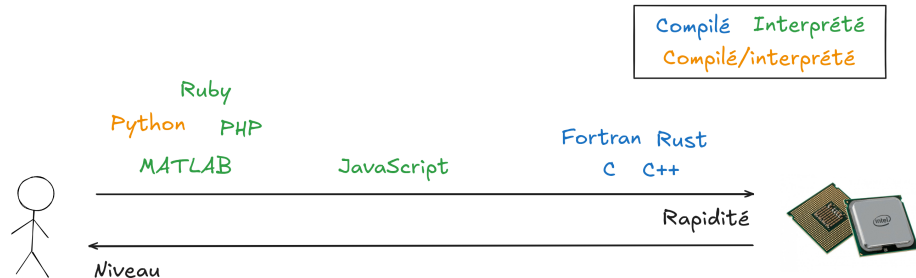
```
1 def distance_euclidienne(a, b):
2     total = 0
3     for i in range(len(a)):
4         total += (a[i] - b[i])**2
5     return total**0.5
6
7 a = [1., 2.]
8 b = [3., 4.]
9 print(distance_euclidienne(a, b))
10
11 a_arr = np.array(a)
12 b_arr = np.array(b)
13 np.sqrt(np.sum(np.square(a_arr - b_arr)))
```

Quelques éléments sur Python



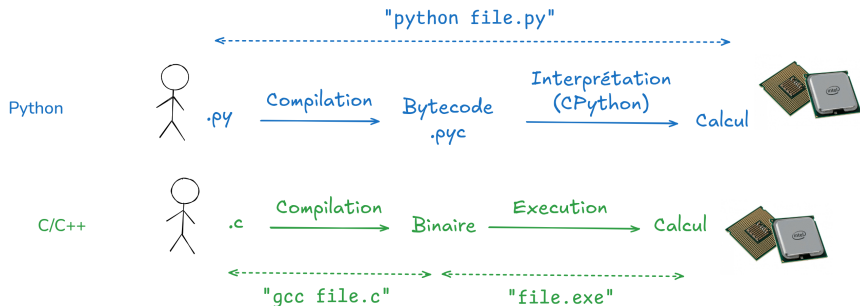
LANGAGES COMPILÉS VS. INTERPRÉTÉS

- ▶ Langages **compilés** (C++, Fortran, etc)
 - Le code humain est transformé en code binaire compréhensible par le CPU (**compilation**)
 - Puis le code est exécuté *i.e.* les calculs sont effectués à proprement dit (**exécution**)
- ▶ Langages **interprétés** (Python, Matlab, etc)
 - Le code humain est lu par un programme tiers (**interpréteur**) qui lit chacune des lignes et l'exécute
 - Généralement **plus lents** que les langages compilés du fait de l'intermédiaire
- ▶ On distingue différents **niveaux** de langage informatique
 - **Langage bas niveau** : proche du langage machine
 - **Langage haut niveau** : proche du langage humain
 - Un langage haut niveau est généralement plus **permissif**, on ne gère pas directement la mémoire et demande moins de connaissances informatiques sous-jacentes



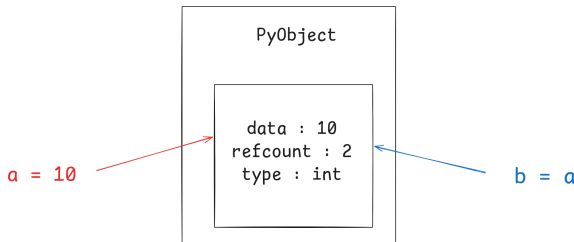
CAS DE PYTHON

- ▶ En fait, Python n'est **pas vraiment** un langage interprété
- ▶ Le code humain est d'abord traduit en un langage intermédiaire (**bytecode**) : c'est une phase de **compilation**
- ▶ Le **bytecode** (.pyc dans `__pycache__`) est indépendant de l'OS, ce n'est pas du code compréhensible par le CPU
- ▶ Le bytecode est **interprété** par l'interpréteur **CPython**
- ▶ Cette interprétation, instruction par instruction, fait de Python un langage lent
- ▶ Numpy appelle du code C compilé : l'interpréteur n'intervient **qu'une fois** et pas à chaque élément



GESTION MÉMOIRE ET GARBAGE COLLECTOR

- ▶ En Python, l'utilisateur ne gère pas lui même la mémoire
- ▶ Chaque objet Python (`int`, `float`, `bool`, etc) est un `PyObject`
- ▶ Il comporte sa donnée mais aussi un **compteur de référence**
- ▶ Lorsque ce compteur tombe à zéro, la mémoire associée est **immédiatement** libérée⁷
- ▶ Ce système est pratique mais possède un inconvénient : les objets sont plus lourds que leur valeur brute
- ▶ D'où l'écart vu précédemment : un `int` Python pèse 28 octets (8 pour un `np.int64`)



7. Dans le cadre de références cycliques, un mécanisme supplémentaire est utilisé (**garbage collector**)

MUTABILITÉ

- ▶ Python (et d'autres langages !) distingue les types **mutables** et **immuables**
- ▶ Type **immutable** (`int`, `float`, `str`, etc)
 - On ne peut changer leur valeur
 - Si cela doit arriver, un nouvel objet est créé
- ▶ Type **mutable** (`list`, `set`, etc)
 - Peuvent être modifiables sans nouvelle création d'objet
- ▶ L'immutabilité se justifie de plusieurs manières : sécurité, performances, etc

```
1 a = 1
2 print("id(a) (avant modif) : ", id(a))
3 a += 1
4 print("id(a) (après modif) : ", id(a))
5
6 L = [1., 2]
7 print("id(L) (avant modif) : ", id(L))
8 L.append(3)
9 print("id(L) (après modif) : ", id(L))
```

```
id(a) (avant modif) : 11277928
id(a) (après modif) : 11277960
id(L) (avant modif) : 132004219807808
id(L) (après modif) : 132004219807808
```

EXERCICE : MUTABILITÉ 1

- ▶ Prendre deux entiers a et b et en faire une liste $L = [a, b]$
- ▶ Faire une fonction f qui prend en argument une liste et qui en réaffecte le premier élément
- ▶ Faire $f(L)$ et afficher L et a
- ▶ Comment expliquer ce qui s'est passé ?

EXERCICE : MUTABILITÉ 1

- ▶ Prendre deux entiers a et b et en faire une liste $L = [a, b]$
- ▶ Faire une fonction f qui prend en argument une liste et qui en réaffecte le premier élément
- ▶ Faire $f(L)$ et afficher L et a
- ▶ Comment expliquer ce qui s'est passé ?

```
1 a = 1
2 b = 2
3 def f(liste):
4     liste[0] = 10
5 L = [a, b]
6 f(L)
7 print(L[0], a)
```

10 1

EXERCICE : MUTABILITÉ 2

- ▶ Faire un tuple composé d'un `int` et d'une `list`
- ▶ Incrémenter le premier élément ? Que se passe-t-il ?
- ▶ Ajouter un élément au second élément (la liste) ? Que se passe-t-il ?
- ▶ Expliquer ce qui se passe

EXERCICE : MUTABILITÉ 2

- ▶ Faire un tuple composé d'un `int` et d'une `list`
- ▶ Incrémenter le premier élément ? Que se passe-t-il ?
- ▶ Ajouter un élément au second élément (la liste) ? Que se passe-t-il ?
- ▶ Expliquer ce qui se passe

```
1 t = (1, [2., 3])  
2 # t[0] += 1 # Erreur  
3 t[1].append(4) # Pas d'erreur  
4 print(t)
```

```
(1, [2.0, 3, 4])
```


CONCLUSION

- ▶ Pourquoi Numpy est plus rapide que Python natif ?
 - ▶ Les listes Python pointent vers des objets **dispersés** en mémoire, chacun avec un **overhead** (PyObject)
 - ▶ Les arrays Numpy sont **contigus** en mémoire : maximisation des cache hits, accès direct via les **strides**
 - ▶ Numpy **vectorise** les calculs : l'interpréteur Python n'intervient qu'une fois contrairement à une itération sur les éléments d'une liste
- ⇒ On explique enfin le facteur $\sim 68\times$ observé en introduction !